PROJECT PROPOSAL: SACCO MICRO-LOAN REPAYMENT AND ARREARS EARLY-WARNING MONITOR

Course: INSY 8311 – Database Development with PL/SQL

Instructor: Eric Maniraguha

Student Name: Mugisha Honore

Student ID: 20251SEN249

GitHub Repository:https://github.com/MugishaHonore2002/mugisha_20251SEN249_saccoRisk

1. Problem Statement

Savings and Credit Cooperatives (SACCOs) are a main source of small loans for farmers, traders and households in Rwanda who are not served by commercial banks. Many SACCOs still track loans, instalments and guarantors in notebooks or spreadsheets. As a result, late payments are discovered only after a loan is already seriously overdue, loan officers cannot easily see which members are drifting into arrears, and new loans can be approved for members who already have unpaid balances. These weaknesses raise the portfolio-at-risk (PAR) and threaten the savings of every member.

There is a need for a database that records each loan and repayment accurately, scores repayment risk, and warns officers early, while enforcing lending rules in the database itself so that they cannot be bypassed.

2. Proposed Solution

This project builds a SACCO Micro-Loan Repayment and Arrears Early-Warning Monitor in an Oracle Pluggable Database ([your PDB name]). The system stores members, loan products, applications, loans, guarantors, repayment schedules and repayments. PL/SQL procedures apply each payment to the oldest unpaid instalment first. A scoring function rates each member's repayment risk, and a scheduled routine classifies overdue loans into 30, 60 and 90-day PAR buckets and raises alerts. Triggers block loans to members in arrears and record every change to repayment data in an audit log.

3. Data Source

The project will use the Kiva Loans dataset on Kaggle [add exact link, size and licence after checking], which contains real, anonymised microfinance loans with amounts, sectors, countries (including Rwanda), terms and repayment intervals. Records will be filtered and reshaped into SACCO tables; member names and identifiers will be replaced by generated codes so that no real person is identified. Fields the dataset lacks (such as instalment dates) will be derived with PL/SQL and documented in the repository. The data will be loaded with SQL*Loader and PL/SQL routines after cleaning.

4. Planned Tables (3NF Schema)

1. **BRANCH** (branch_id PK, branch_name, district) – SACCO branches.

2. **MEMBER** (member_id PK, member_code, branch_id FK, join_date, status) – anonymised members.
3. **LOAN_PRODUCT** (product_id PK, product_name, interest_rate, max_amount, max_term_months) – loan types.
4. **LOAN_APPLICATION** (application_id PK, member_id FK, product_id FK, amount_requested, apply_date, decision) – requests.
5. **LOAN** (loan_id PK, application_id FK, amount_approved, disbursed_date, status) – approved loans.
6. **GUARANTOR** (guarantor_id PK, loan_id FK, member_id FK, amount_guaranteed) – members backing a loan.
7. **REPAYMENT_SCHEDULE** (schedule_id PK, loan_id FK, due_date, amount_due, status) – instalments.
8. **REPAYMENT** (repayment_id PK, schedule_id FK, paid_date, amount_paid) – payments received.
9. **RISK_ASSESSMENT** (assessment_id PK, member_id FK, risk_score, risk_band, assessed_at) – scores.
10. **ARREARS_ALERT** (alert_id PK, loan_id FK, days_overdue, par_bucket, raised_at, resolved) – early warnings.
11. **AUDIT_LOG** (audit_id PK, table_name, action, old_value, new_value, user_name, action_time) – change history.

5. Planned PL/SQL Features

- **Package PKG_SACCO_LOANS:** groups loan approval, repayment and alert logic in one unit.
- **Procedure PRC_RECORD_REPAYMENT:** applies a payment to the oldest unpaid instalment and updates its status; raises a custom exception for overpayment or a closed loan.
- **Procedure PRC_APPROVE_LOAN:** checks product limits and guarantor cover before creating a loan and its schedule.
- **Function FNC_CALC_RISK_SCORE:** returns a 0–100 score from late-payment history, loan size relative to savings and guarantor cover.
- **Cursor (explicit):** loops over overdue instalments to compute days overdue and insert PAR alerts; a second cursor produces the monthly arrears report.
- **Trigger TRG_BLOCK_LOAN_IN_ARREARS:** prevents a new loan for a member with unresolved arrears.
- **Trigger TRG_AUDIT_REPAYMENT:** writes old and new values of repayment changes to AUDIT_LOG.
- **Exceptions:** handle invalid amounts, missing members and duplicate payments.

6. Planned Innovation

- **Category:** User interface (Option 2), with an optional analytics dashboard.
- **Tool:** Oracle APEX connected to the Oracle database.
- **Functionality:** A loan officer searches for a member, records a repayment through a form that calls PRC_RECORD_REPAYMENT, and views live arrears alerts and risk scores. All business rules stay in PL/SQL; APEX only displays and submits data.

7. References (APA)

- Access to Finance Rwanda. (2020). *FinScope Rwanda 2020: Consumer survey report*. https://microdata.statistics.gov.rw/index.php/catalog/89/download/832
- Apache Software Foundation. (n.d.). *Apache Fineract* [Computer software]. https://fineract.apache.org/
- Kasina, M. M., Kihoro, J. M., & Kibet, A. (2025). Performance of different machine learning algorithms for credit risk classification. *Journal of Data Analysis and Information Processing, 13*(4). https://doi.org/10.4236/jdaip.2025.134029
- Kiva. (n.d.). *Data science for good: Kiva crowdfunding* [Data set]. Kaggle. https://www.kaggle.com/kiva/data-science-for-good-kiva-crowdfunding
- Koffi, C. H. A., Djeundje, V. B., & Menoukeu Pamen, O. (2024). *Quantifying socio-temporal effects of loan delinquency drivers in microfinance* [Preprint]. arXiv. https://arxiv.org/abs/2410.13100
- Oracle Corporation. (2025). *Oracle Database PL/SQL language reference, 19c* (E96448-08). https://docs.oracle.com/en/database/oracle/oracle-database/19/lnpls